COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 09-04-2026 | **DATE SUBMITTED:** | 09-08-2026 |
| **NAME:** | GUARIN, JAN MIKAELA C. | **INSTRUCTOR:** | ENGR. JIMLORD QUEJADO |

## ACTIVITY NO. 7.1 Sorting Algorithms Part 1

### 1. PROCEDURE OUTPUT

**Table 7.1 - Array of Values for Sort Algorithm Testing**

**Code + Console Screenshot**

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main()
{

    // create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];

    // use time(0) as seed for random values
    srand(time(0));

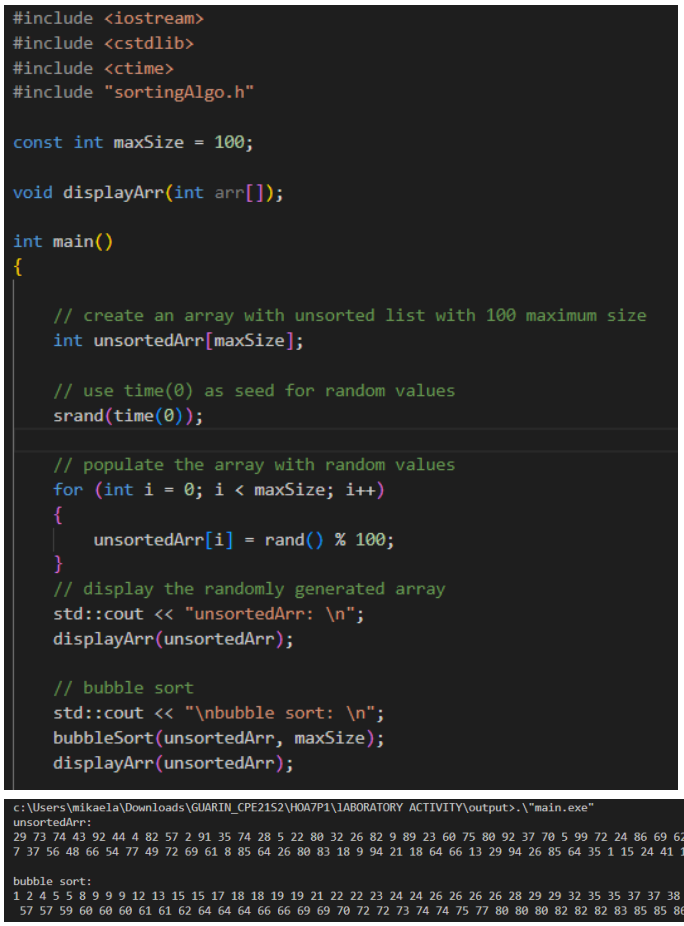
    // populate the array with random values
    for (int i = 0; i < maxSize; i++)
    {
        unsortedArr[i] = rand() % 100;
    }
    // display the randomly generated array
    std::cout << "unsortedArr: \n";
    displayArr(unsortedArr);
```

```
c:\Users\mikaela\Downloads\GUARIN_CPE21S2\HOA7P1\LABORATORY ACTIVITY\output>.\"main.exe"
unsortedArr:
83 16 42 65 25 29 11 14 63 88 30 27 5 37 35 72 36 27 22 41 13 90 74 88 93 40 2 75 65 21 86 11 66 11 99 5 59 73 42 84 38 94 6 31 69 65 4 77 14 52 7
8 6 97 34 91 36 52 2 50 6 47 21 25 79 97 45 16 15 14 71 82 9 35 37 16 24 83 20 64 27 56 81 67 42 33 79 77 8 53 53 40 50 72 12 24 1 95 19 90 39
```

**Observations**

For this task, I created an integer array named unsortedArr with a maximum capacity of 100 elements using the maxSize constant. I used srand(time(0)) to initialize the random number generator and applied rand() % 100 to generate values between 0 and 99 that were stored inside the array. I also created a displayArr() function that allowed me to print each element of the array using a loop.

**Table 7.2 - Bubble Sort Technique**

| | |
|---|---|
| **Code + Console Screenshot** |  |
| **Observations** | For the Bubble Sort implementation, I developed a template function called bubbleSort() that accepts the array and its size as parameters. I used nested loops to repeatedly compare adjacent elements in the array. Whenever I found that the current element was greater than the next element, I swapped their positions using a temporary variable. After multiple comparisons and exchanges, I observed that the array values were successfully rearranged in ascending order. |

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main()
{

    // create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];

    // use time(0) as seed for random values
    srand(time(0));

    // populate the array with random values
    for (int i = 0; i < maxSize; i++)
    {
        unsortedArr[i] = rand() % 100;
    }
    // display the randomly generated array
    std::cout << "unsortedArr: \n";
    displayArr(unsortedArr);

    // bubble sort
    std::cout << "\nbubble sort: \n";
    bubbleSort(unsortedArr, maxSize);
    displayArr(unsortedArr);
```

```
c:\Users\mikaela\Downloads\GUARIN_CPE21S2\HOA7P1\1ABORATORY ACTIVITY\output>.\"main.exe"
unsortedArr:
29 73 74 43 92 44 4 82 57 2 91 35 74 28 5 22 80 32 26 82 9 89 23 60 75 80 92 37 70 5 99 72 24 86 69 62 42 94 59 57 49 49 26 42 95 19 90 47 96 22 1
7 37 56 48 66 54 77 49 72 69 61 8 85 64 26 80 83 18 9 94 21 18 64 66 13 29 94 26 85 64 35 1 15 24 41 19 97 53 82 40 61 12 15 48 9 60 60 38 87 87

bubble sort:
1 2 4 5 5 8 9 9 9 12 13 15 15 17 18 18 19 19 21 22 22 23 24 24 26 26 26 26 28 29 29 32 35 35 37 37 38 40 41 42 42 43 44 47 48 48 49 49 49 53 54 56
 57 57 59 60 60 60 61 61 62 64 64 64 66 66 69 69 70 72 72 73 74 74 75 77 80 80 80 82 82 82 83 85 85 86 87 87 89 90 91 92 92 94 94 94 95 96 97 99
```

**Table 7.3 - Selection Sort Algorithm**

**Code + Console Screenshot**

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main()
{

    // create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];

    // use time(0) as seed for random values
    srand(time(0));

    // populate the array with random values
    for (int i = 0; i < maxSize; i++)
    {
        unsortedArr[i] = rand() % 100;
    }
    // display the randomly generated array
    std::cout << "unsortedArr: \n";
    displayArr(unsortedArr);

    // bubble sort
    // std::cout << "\nbubble sort: \n";
    // bubbleSort(unsortedArr, maxSize);
    // displayArr(unsortedArr);

    // selection sort
    std::cout << "\nselection sort : \n ";
    selectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);
```

```cpp
template <typename T>
// array, current position, size of array
int routineSmallest(T arr[], int K, const int arrSize)
{
    int position = K;
    T smallestElem = arr[K];

    for (int j = K + 1; j < arrSize; j++)
    {
        if (arr[j] < smallestElem)
        {
            smallestElem = arr[j];
            position = j;
        }
    }

    return position;
}

template <typename T>
void selectionSort(T arr[], size_t arrSize)
{
    int POS;

    for (int i = 0; i < arrSize - 1; i++)
    {
        POS = routineSmallest(arr, i, arrSize);

        // swapping
        T temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
    }
}
```

```
c:\Users\mikaela\Downloads\GUARIN_CPE21S2\HOA7P1\1ABORATORY ACTIVITY\output>.\"main.exe"
unsortedArr:
65 40 78 67 46 67 58 61 82 27 2 59 70 81 4 52 19 12 52 72 76 27 54 74 99 80 18 5 16 19 33 90 53 15 50 35 5 88 54 30 11 30 7 15 10 89 15 96 23 26 9
1 20 82 72 33 90 29 34 59 31 69 8 77 9 14 72 50 33 69 96 42 35 4 81 22 40 38 10 84 89 4 0 33 34 98 56 92 9 62 40 5 93 46 94 2 1 92 67 1 49

selection sort :
 0 1 1 2 2 4 4 4 5 5 5 7 8 9 9 10 10 11 12 14 15 15 15 16 18 19 19 20 22 23 26 27 27 29 30 30 31 33 33 33 33 34 34 35 35 38 40 40 40 42 46 46 49 5
0 50 52 52 53 54 54 56 58 59 59 61 62 65 67 67 67 69 69 70 72 72 72 74 76 77 78 80 81 81 82 82 84 88 89 89 90 90 91 92 92 93 94 96 96 98 99
```

| | |
|---|---|
| **Observations** | For the Bubble Sort implementation, I developed a template function called bubbleSort() that accepts the array and its size as parameters. I used nested loops to repeatedly compare adjacent elements in the array. Whenever I found that the current element was greater than the next element, I swapped their positions using a temporary variable. After multiple comparisons and exchanges, I observed that the array values were successfully rearranged in ascending order. |

**Table 7.4 - Insertion Sort Algorithm**

| | |
|---|---|
| **Code + Console Screenshot** | |

```
    // insertion sort
    std::cout << "\nInsertion Sort: \n";
    insertionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    return 0;
}

void displayArr(int arr[])
{
    for (int i = 0; i < maxSize; i++)
    {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
}
```

```
template <typename T>
void insertionSort(T arr[], size_t arrSize)
{
    int K = 1, J;
    T temp;

    while (K < arrSize)
    {
        temp = arr[K];
        J = K - 1;

        while (J >= 0 && temp <= arr[J])
        {
            arr[J + 1] = arr[J];
            J--;
        }

        arr[J + 1] = temp;
        K++;
    }
}
#endif
```

```
unsortedArr:
98 69 85 41 30 52 41 66 57 4 69 42 81 26 51 51 55 49 54 35 48 66 60 3 10 36 87 60 34 41 31 93 23 23 58 73 51 47 19 35 96 61 51 67 10 22 13 11 79 7
8 3 46 36 26 1 95 22 74 82 41 71 50 37 98 13 90 70 49 27 21 33 21 83 34 29 35 76 38 13 18 10 45 87 93 90 33 98 16 74 67 86 45 32 63 24 98 62 79 20
 58

Insertion Sort:
1 3 3 4 10 10 10 11 13 13 16 18 19 20 21 21 22 22 23 23 24 26 26 27 29 30 31 32 33 33 34 34 35 35 35 36 36 37 38 41 41 41 41 42 45 45 46 47 48
49 49 50 51 51 51 51 52 54 55 57 58 58 60 60 61 62 63 66 66 67 67 69 69 70 71 73 74 74 76 78 79 79 81 82 83 85 86 87 87 90 90 93 93 95 96 98 98 98
 98
```

**Observations**

For the Insertion Sort implementation, I created the insertionSort() template function to arrange the elements by inserting each value into its proper position. I started from the second element of the array, stored its value temporarily, and compared it with the previous elements. When larger values were encountered, I shifted them to the right until the correct location for the stored value was found. After completing the process, I observed that the

| | |
|---|---|
| | output displayed the array in ascending order, showing that the insertion sort algorithm worked correctly. |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

| **PSUEDOCODE** |
|---|
| START<br>Create an array to store 100 vote values<br>Generate random vote numbers from 1 to 5 and store them in the array<br>Display the generated vote records<br>Arrange the vote values using Selection Sort<br>Check each sorted value and add it to the corresponding candidate count<br>Compare the total votes of all candidates<br>Display the vote results and the candidate with the most votes<br>END |

**Code + Console Screenshot**

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "suppAlgo.h"

const int maxSize = 100;

void displayArray(int arr[])
{
    for (int i = 0; i < maxSize; i++)
    {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
}

int main()
{
    std::string candidates[5] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"};

    int votes[maxSize];

    int totalVotes[5] = {0};

    srand(time(0));

    // create random votes
    for (int i = 0; i < maxSize; i++)
    {
        votes[i] = (rand() % 5) + 1;
    }

    std::cout << "Original Votes:\n";
    displayArray(votes);

    // arrange votes
    selectionSort(votes, maxSize);
    std::cout << "\nSorted Votes:\n";
    displayArray(votes);

    // count each candidate vote
    for (int i = 0; i < maxSize; i++)
    {
        int candidateNumber = votes[i];

        totalVotes[candidateNumber - 1]++;
    }

    std::cout << "\nCandidate Results:\n";

    int highestVote = totalVotes[0];
    int winningCandidate = 0;

    for (int i = 0; i < 5; i++)
    {
```

```cpp
    {
        int candidateNumber = votes[i];

        totalVotes[candidateNumber - 1]++;
    }

    std::cout << "\nCandidate Results:\n";

    int highestVote = totalVotes[0];
    int winningCandidate = 0;

    for (int i = 0; i < 5; i++)
    {
        std::cout << candidates[i]
                  << ": "
                  << totalVotes[i]
                  << " votes"
                  << std::endl;

        if (totalVotes[i] > highestVote)
        {
            highestVote = totalVotes[i];
            winningCandidate = i;
        }
    }

    std::cout << "\nWinner:\n";
    std::cout << candidates[winningCandidate]
              << " with "
              << highestVote
              << " votes."
              << std::endl;

    return 0;
}
```

```
#ifndef SUPPALGO_H
#define SUPPALGO_H

#include <cstddef>

template <typename T>
int findSmallest(T arr[], int start, int size)
{
    int smallest = start;

    for (int i = start + 1; i < size; i++)
    {
        if (arr[i] < arr[smallest])
        {
            smallest = i;
        }
    }

    return smallest;
}

template <typename T>
void selectionSort(T arr[], size_t size)
{
    for (int i = 0; i < size - 1; i++)
    {
        int smallestIndex = findSmallest(arr, i, size);

        T holder = arr[i];
        arr[i] = arr[smallestIndex];
        arr[smallestIndex] = holder;
    }
}

#endif
```

```
C:\Users\mikaela\Downloads\GUARIN_CPE21S2\HOA7P1\LABORATORY ACTIVITY\output>.\"suppmain.exe"
Original Votes:
2 4 2 3 5 1 5 4 1 2 4 2 5 1 2 5 1 3 4 3 2 2 3 5 5 1 2 1 4 3 1 3 3 5 5 1 4 1 5 1 1 4 3 4 3 4 4 1 1 3 5 2 5 2 4 4 2 3 2 3 1 4 1 5 4 1 2 2 5 3 3 1 1 2 2 3 4 2 3 4 2 4 5 2 2 4 1 3 2 5 4 3 2 1 4 3 3 3 2 5

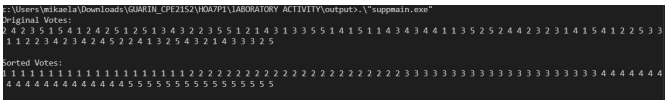
Sorted Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Candidate Results:
Bo Dalton Capistrano: 20 votes
Cornelius Raymon Agustin: 23 votes
Deja Jayla Banaga: 21 votes
Lalla Brielle Yabut: 20 votes
Franklin Relano Castro: 16 votes

Winner:
Cornelius Raymon Agustin with 23 votes.
```

| | |
|---|---|
| **Question/ Justification** | Yes, the developed vote counting algorithm was effective because it was able to correctly organize the generated vote data, count the number of votes received by each candidate, and determine the candidate with the highest votes. By using the Selection Sort algorithm, the vote values were arranged in ascending order, which made it easier to group and count similar vote entries. The results produced by the algorithm matched the manual counting, confirming that the program accurately processed the votes. Therefore, the algorithm successfully performed its intended purpose of analyzing the vote data and identifying the winning candidate. |

| **Output Console Showing Sorted Array** | **Manual Count** | **Count Result of Algorithm** |
|---|---|---|




| | |
|---|---|
| Candidate 1 = 20 votes | Candidate 1 = 20 votes |
| Candidate 2 = 23 votes | Candidate 2 = 23 votes |
| Candidate 3 = 21 votes | Candidate 3 = 21 votes |
| Candidate 4 = 20 votes | Candidate 4 = 20 votes |
| Candidate 5 = 16 votes | Candidate 5 = 16 votes |

## CONCLUSION

For this laboratory activity, I learned more about the implementation of sorting algorithms such as bubble sort, selection sort, and insertion sort using C++. I was able to understand how each algorithm arranges data through different processes like comparing, swapping, and shifting elements. At first, understanding the logic behind these algorithms was challenging, but practicing the code and applying selection sort in the supplementary activity helped me improve my understanding. The results of the program showed how sorting can make data processing easier, especially when organizing votes and determining the candidate with the highest number of votes. In conclusion, this activity helped me enhance my programming skills and recognize the importance of selecting an appropriate algorithm for solving problems efficiently.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

COMPUTER ENGINEERING

QUEZON CITY

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.